Mobile Cloud, Fog Computing, IoT, Docker & Kubernetes

MCQ Concept Mastery Notes

Explained for Absolute Beginners — Step by Step

Part A — Mobile Cloud & Fog/Edge Computing

Part B — Docker, Kubernetes & Green Cloud

April 18, 2026 • Beginner-Friendly with Analogies and Exam Answers

Contents

Part A — Mobile Cloud & Fog/Edge Computing

1.1 Q1 — Why Does Mobile Cloud Gaming Lag When Switching Networks?

Mobile Cloud Gaming — How It Actually Works

In mobile cloud gaming (Google Stadia, Xbox Cloud Gaming, NVIDIA GeForce NOW), your phone is **not** running the game. A powerful server far away in the cloud runs the game. Your phone only:

- Sends your inputs (taps, swipes, joystick moves) to the server
- Receives compressed video frames back from the server
- Displays those frames on your screen

This means the **internet connection is the most critical link** in the entire experience.

Real-Life Analogy

Think of watching a live cricket match streamed over the internet:

- **Your phone** = TV screen displaying the match
- **Cloud server** = The actual stadium computing all the gameplay
- **Internet connection** = The broadcasting cable linking them

When you switch from Wi-Fi to 4G, that cable changes. If the new connection is weaker or unstable, the broadcast becomes choppy, frames drop, and the game lags. Nothing changed in the game itself — only the connection quality did.

What happens technically when you switch networks:

- **Latency increases:** Delay between your action and server response grows
- **Bandwidth drops:** Fewer video frames reach your phone per second
- **Packet loss:** Some data gets dropped in transit, causing stuttering
- **Jitter:** Inconsistent packet timing makes the game feel unresponsive

Correct Answer: High dependency on Wide-Area Network (WAN) conditions

Mobile cloud gaming requires a **consistently fast, stable, low-latency internet connection at all times**. Even a 100ms increase in latency makes the game feel unresponsive. No amount of phone hardware can fix a bad internet connection, because the actual game runs on a remote server.

Memory Trick

Cloud gaming shifts the bottleneck from your phone's CPU to your internet connection. Good connection = smooth game. Bad connection = unplayable lag.

1.2 Q2 — Why Do We Use a Cloudlet?

What is a Cloudlet?

A **cloudlet** is a small, powerful server placed **physically close to users** — inside a building, campus, or city area. It is a "mini cloud" that handles computation locally instead of sending data to a distant data centre.

Stadium AR App Scenario

You are in a cricket stadium using an AR app that shows live player stats when you point your camera at a player.
Without a cloudlet (far cloud): Phone sends image → cloud server overseas → processed → result returns. Total: 200–500ms. Stats appear with a noticeable lag.
With a cloudlet in the stadium: Phone sends image → cloudlet 10 metres away → processed instantly → result in under 10ms. Stats appear truly in real-time.

Real-Life Analogy

Ordering food:
- Restaurant in another city → hours away (far cloud)
- Restaurant next door → arrives in minutes (cloudlet)

The cloudlet is the "restaurant next door" of cloud computing.

Correct Answer: To provide low-latency computing close to users

Cloudlets eliminate the distance problem of cloud computing. For AR/VR, gaming, live video, and face recognition — applications that need instant responses — the cloudlet nearby makes the experience genuinely real-time.

Memory Trick

Cloudlet = Mini Cloud + Close Location = Low Latency. Use for: AR/VR, real-time gaming, live processing, face recognition.

1.3 Q3 — What is NOT Useful for Deciding Offloading?

What is Computation Offloading?

Offloading means deciding whether your phone should process a task itself, or send it to the cloud. The system checks real-time factors to decide: Is it worth sending this task to the cloud right now?

Useful factors for offloading decisions:

- Current internet speed and bandwidth
- Battery level of the device
- Task complexity and estimated processing time
- Current CPU load on the device

- Network latency to the cloud

Correct Answer: Number of apps installed on the device

How many apps are installed (not currently running) is **completely irrelevant** to offloading decisions. Installed-but-inactive apps do not consume CPU, battery, or network resources. The decision is based on **real-time resource status**, not software inventory.

Memory Trick

Relevant: Battery + Network speed + Task size + Current load. Not relevant: Number of installed apps, phone colour, screen brightness.

1.4 Q4 — What is the Server Inside the Campus Cafeteria?

The Scenario

A university installs a small powerful server inside the cafeteria. Students using AR apps connect to this local server for fast processing. No data leaves the campus to a distant cloud.

Option	Why It Fits / Does Not Fit
Public Cloud	Far-away data centre, not inside the cafeteria
Satellite	Used for communication only, not local processing
Personal Hotspot	Shares internet only, does not process tasks
Cloudlet	Small powerful server placed close to users — exactly this!

Correct Answer: A Cloudlet placed close to end users

The cafeteria server matches the exact cloudlet definition: physically close to users, processes tasks locally, provides low-latency responses.

Memory Trick

Quick Rule: Small processing server inside a building near users = Cloudlet. Amazon/Google data centre = Cloud.

1.5 Q5 — Biggest Security Concern in Mobile Cloud Banking

The Data Journey in Mobile Banking

When you use a banking app, sensitive data travels: Phone → Wi-Fi/4G → Internet → Cloud → Bank Server. At every hop, data could potentially be intercepted or stolen. Sensitive data at risk: login credentials, account numbers, balance, transaction details, personal identity.

Correct Answer: Confidentiality of sensitive data during transmission and processing

Confidentiality = only authorised people can read the data. In mobile cloud banking, this must hold in transit (HTTPS/TLS encryption), during processing (access controls), and at rest (encrypted databases). A breach = stolen passwords, drained accounts, identity fraud.

Why Other Options Are Wrong

Screen size, UI design, icon appearance, phone colour — none of these are security concerns. Security is always about **the data**, not the device's physical properties.

Memory Trick

Banking + Cloud = protect the data at all times. Confidentiality in transit AND in processing AND in storage.

1.6 Q6 — Why is Fog Computing Better Than Cloud for Real-time Apps?

Cloud vs Fog — The Core Difference

- **Cloud:** Data travels to a distant data centre (100–500ms round-trip). Fine for tasks that can wait.
- **Fog:** Processing happens at nearby intermediate nodes (1–20ms). Essential for time-critical tasks.

Heart Monitor Emergency

A patient wears a cardiac monitoring device. ECG data must trigger an emergency alert if abnormal.
Cloud-only: ECG → cloud (200ms) → analysis → alert back (200ms) = 400ms total. Potentially fatal delay.
Fog Computing: ECG → fog node in hospital (5ms) → instant alert. Life saved.

Correct Answer: Better support for real-time applications through reduced latency

For safety-critical or life-critical applications (healthcare, autonomous vehicles, smart factory safety systems), eliminating that cloud round-trip delay is not optional — it is essential for correct operation.

Memory Trick

Fog = Cloud's faster local cousin. Response needed in milliseconds → Fog. Can wait seconds → Cloud is fine.

1.7 Q7 — Why Combine 5G With Edge/Fog Computing?

Two Types of Delay in IoT Systems

Real-time IoT applications face two separate delays:
1. **Network delay** — time for data to travel from device to server (5G reduces this)
2. **Processing delay** — time for the server to compute the result (Edge/Fog reduces this)

5G and Edge are complementary — each solves a *different* delay problem.

Real-Life Analogy

- **5G = Ultra-fast motorway** from your device to the nearest processing point
- **Edge node = Service station right at the motorway exit** — no need to travel to the city centre

Fast road (5G) + nearby destination (Edge) = near-zero total travel time.

Technology	Reduces	How
5G	Network / transmission delay	Fast wireless, low air-interface latency
Edge/Fog	Processing / computation delay	Servers physically near devices
5G + Edge	Both types of delay	Combined: near-zero total latency

Correct Answer: Edge reduces processing delay + 5G reduces network delay, achieving ultra-low total latency

Perfect for: autonomous vehicles, remote surgery, industrial automation, smart grids.

Why Other Options Are Wrong

- "5G guarantees zero data loss" → False. 5G reduces packet loss but does not eliminate it.
- "Edge removes need for hardware" → False. Edge nodes are physical hardware.
- "Data becomes publicly available" → Completely unrelated.

1.8 Q8 — How to Process Huge Satellite Images Faster?

The Scale Problem

A single high-resolution satellite image can be several gigabytes. Processing thousands of such images for flood mapping, crop monitoring, or disaster response would take weeks on a single processor.

Real-Life Analogy

Painting a 100-metre mural:
- 1 painter (sequential): takes months
- 100 painters each doing 1 metre (parallel): done in days

Divide the image into thousands of tiles, process all tiles simultaneously on cloud workers. Total time: hours → minutes.

Correct Answer: Partition data into smaller blocks and process in parallel

Split large images into tiles → assign each tile to a cloud worker (MapReduce, Spark, Dask) → all workers process simultaneously → combine results. This is the standard approach in geospatial cloud computing.

Why Other Options Are Wrong

- Better screen resolution → affects display, not processing speed
- Single-thread processing → makes it *slower*, not faster
- Shorter filenames → completely irrelevant to computational performance

1.9 Q9 — Biggest Challenge in Geospatial Cloud Systems

What Geospatial Systems Handle

Geospatial cloud systems work with satellite imagery (GBs per image), LiDAR point clouds (3D terrain scans), GPS streams, and aerial surveys. These datasets are among the largest in the world, growing constantly and requiring complex analysis.

Why this is uniquely challenging:

- **Storage:** A satellite captures terabytes per day. Years of data = petabyte-scale storage.
- **Computation:** Object detection, change detection, and 3D reconstruction algorithms are computationally intense.
- **Data movement:** Transferring huge images from storage to processing nodes is itself a bottleneck.
- **Time pressure:** Disaster response needs fast results on enormous datasets simultaneously.

Correct Answer: Very high computation and storage requirements

The combination of enormous data volumes + complex analysis algorithms + time pressure makes geospatial systems one of the most demanding domains. Large-scale distributed cloud infrastructure is essential — no single machine can handle it.

Why Other Options Are Wrong

- Lack of user interest → Geospatial data is in extremely high demand
- Cannot run GIS on cloud → Cloud GIS tools exist and work well (Google Earth Engine, AWS)
- Standards removed → OGC standards still exist and are actively used

1.10 Q10 — Best Architecture for Real-time Healthcare Monitoring

The Wearable ECG Scenario

A patient wears a smart cardiac monitor. If it detects an abnormality, an alert must trigger within milliseconds.
Pure Cloud: ECG data → internet → cloud (300–500ms round-trip) → alert back. Potentially too late in a cardiac event.
Batch Processing: Upload once per day. Patient could need help hours before the batch runs.
Fog/Edge + 5G: ECG → fog node in hospital ward (5ms) → instant alert via 5G. Fast enough to save a life.

Correct Answer: Fog/Edge processing + low-latency network like 5G

- **Edge/Fog:** Processes patient data locally in the hospital. Decisions in milliseconds. No cloud round-trip.
- **5G:** Fast reliable wireless for the wearable to reach the edge node.
- **Cloud (optional):** Long-term storage, AI model training — tasks tolerating delay.

Real-Life Analogy

- Doctor in the same room: instant response (Edge + 5G)
- Specialist consultant in another city: 30 minutes for advice (Cloud only)

For emergencies, you need help *in the room*.

Part A Quick Recap

Q1: Gaming lag = WAN dependency Q2: Cloudlet = mini nearby cloud
Q3: App count = useless for offloading Q4: Cafeteria server = cloudlet
Q5: Banking risk = confidentiality Q6: Fog = lower latency, better for real-time
Q7: 5G + Edge = fast network + fast compute Q8: Big images = partition + parallel
Q9: Geospatial = huge storage + huge compute Q10: Healthcare = Edge/Fog + 5G

Part B — Docker, Kubernetes & Green Cloud / Sensor Cloud

2.1 Q1 — Docker Command to Show ALL Containers

What is Docker?

Docker lets you run applications inside **containers** — isolated environments containing everything the app needs. Containers can be in two states: **running** (active) or **stopped** (exited but still exists on disk).

Command	What It Shows
`docker ps`	Only **currently running** containers
`docker ps -a`	**ALL** containers (running + stopped + created)

The `-a` flag means `--all`: show everything regardless of state.

Real-Life Analogy

Checking your fridge:
- `docker ps` = only items currently being heated/cooked
- `docker ps -a` = everything in the fridge, used or not

Correct Answer: docker ps -a

Without the `-a` flag, stopped containers are completely invisible. The `-a` flag reveals everything — running, stopped, and created-but-never-started containers.

Memory Trick

docker ps = only running. docker ps -a = ALL. The "a" stands for "all."

2.2 Q2 — Docker Volumes and Data Persistence

The Problem: Containers are Ephemeral by Default

Everything inside a Docker container **disappears when the container stops or is deleted**. This is a serious problem for databases, file uploads, and any application that needs to store data permanently.
Solution: Docker Volumes — storage that exists **outside the container**.

Database Container Example

You run a MySQL container. Users register and their data is saved in the database.
- **Without volume:** Container stops for maintenance → all user data gone!
- **With volume:** Container stops → data on volume is safe. Container deleted →

data still survives. New container attached to same volume → all data is back.

Correct Answer: Docker volumes persist independently of the container lifecycle

Volumes exist beyond any individual container's lifetime. They can be attached to new containers after old ones are deleted, and shared between multiple containers simultaneously.

Why Other Options Are Wrong

- "Volumes stored inside the container" → False. Volumes are external to the container.
- "Volumes deleted when container is removed" → False. This is exactly what volumes prevent.
- "Each container needs its own dedicated volume" → False. Multiple containers can share one volume.

Memory Trick

Container = temporary worker. Volume = permanent safe deposit box. Volume outlives the container.

2.3 Q3 — Containers vs Virtual Machines — OS Sharing

The Fundamental Architecture Difference

- **Virtual Machines:** Each VM has its own *complete, full operating system.* Heavy but strongly isolated.
- **Containers:** All containers on a host share the *same OS kernel.* Lightweight and fast.

A very common real-world setup: Physical server → Hypervisor → Virtual Machine (with Linux OS) → Docker running dozens of containers inside it. All those containers share the VM's Linux kernel.

Real-Life Analogy

- Physical server = large office building
- VM = one floor of the building (own floor plan / OS)
- Containers = cubicles on that floor (share the floor's plumbing/electrical = OS kernel)

Many cubicles (containers) on one floor (VM), all sharing the same infrastructure.

Correct Answer: Multiple containers can share the same OS kernel while running on a VM

One VM running Linux can host dozens or hundreds of containers, all sharing that single Linux kernel. This is why containers are so much lighter and faster than VMs.

Feature	Virtual Machine	Container
OS	Own full OS per VM	Shares host OS kernel
Size	GBs	MBs
Start time	Minutes	Seconds
Isolation	Strong	Weaker
Density per host	Few	Hundreds

2.4 Q4 — What Kubernetes Does NOT Do

What is Kubernetes?

Kubernetes (K8s) is a container **orchestration platform**. It automatically manages large numbers of Docker containers across many machines — like a manager supervising all containers.

What Kubernetes DOES do:

- Restart failed containers automatically

- Schedule containers on appropriate machines based on available resources
- Load balance network traffic across container replicas
- Auto-scale the number of container replicas based on demand
- Perform rolling updates without downtime
- Enable service discovery between containers

Correct Answer: Kubernetes does NOT provide low-level hardware virtualisation via hypervisor extensions

Hardware virtualisation (creating VMs, managing CPU virtualisation extensions like Intel VT-x) is done by **hypervisors** (VMware ESXi, Hyper-V, KVM). Kubernetes operates above the VM/OS layer. It manages containers — it has no concept of hypervisor-level operations.

Real-Life Analogy

- Kubernetes = HR manager (assigns work, manages people/containers)
- Hypervisor = Building infrastructure engineer (manages hardware, power, walls)

The HR manager doesn't wire the building. Different layers, different jobs.

2.5 Q5 — Green Cloud: Energy Saving via VM Consolidation

The Inefficiency Problem in Data Centres

Typical data centres have many servers running at only 20–30% CPU utilisation. Each server draws significant base power even when mostly idle. Enormous electricity is wasted on servers doing almost nothing.

Real-Life Analogy

University classroom analogy:
- 50 students in 5 classrooms (20% full each) → 5 ACs and 5 lighting systems all running = massive waste
- Move all students into 1–2 classrooms, close the rest → only 1–2 ACs and lights needed = big savings

Same principle: pack all VMs onto fewer servers and power off the rest.

Correct Answer: Consolidating VMs onto fewer physical servers and powering down idle machines

VM consolidation process:
- Use live VM migration to move all workloads onto fewer servers
- Power off the now-empty servers completely
- Remaining servers run at higher utilisation (70–90% vs 20–30%)
- Total electricity consumption drops significantly

Why Other Options Are Wrong

- Randomly distributing VMs → inefficient, does not save energy
- Using older servers → older hardware typically consumes more power per unit of work
- Running all servers at maximum speed 24/7 → maximum energy waste

2.6 Q6 — Challenge in Sensor-Cloud with Wireless Sensor Networks

What is a Wireless Sensor Network (WSN)?

A WSN is a collection of small, battery-powered sensor devices (temperature, humidity, motion sensors). These sensors are extremely resource-constrained:
- Very weak CPU (much weaker than a smartphone)
- Tiny memory (kilobytes to a few megabytes)
- Small battery that must last months or years

Cloud systems rely on virtualisation (running hypervisors and VMs), but sensors face a fundamental barrier:

Correct Answer: Sensors cannot support full virtualisation due to limited compute and memory

Full virtualisation requires a hypervisor (significant CPU and RAM), running a full OS or multiple OS instances. A coin-sized temperature sensor simply **cannot run a hypervisor**. Its entire computing capacity might be less than a computer's idle state.

Real-Life Analogy

- Sensor = basic solar-powered calculator
- Cloud server = powerful workstation with 64 cores

You cannot run virtual machines on a calculator. That is the fundamental problem. Solution: Gateway devices bridge the gap between weak sensors and the cloud.

Memory Trick

Sensors are tiny and weak. Full virtualisation needs powerful hardware. Use gateways as the bridge.

2.7 Q7 — Best Architecture for Ultra-Low Latency IoT

Self-Driving Car Obstacle Detection

A self-driving car's sensors detect an obstacle.
Pure Cloud: Sensor data → internet → cloud (500ms) → brake command back (500ms) = 1 second total. Car travels 30 metres at 108 km/h during this delay. Collision.
Edge computing: Sensor data → onboard edge processor (5ms) → brake applied immediately. Safe.

Correct Answer: Use edge/fog computing nodes closer to the devices

Edge computing eliminates the round-trip to a distant cloud server. For applications where latency above 10ms is unacceptable (autonomous vehicles, industrial robotics, real-time control), edge computing is not optional — it is mandatory.
Data does not travel over the wide-area internet. Processing happens in the same vehicle, building, or factory floor. Decisions are made locally in milliseconds.

Memory Trick

Ultra-low latency IoT = Edge/Fog computing. When milliseconds matter, distance kills. Keep processing close.

2.8 Q8 — Why Preprocess Sensor Data Before Sending to Cloud?

The Data Flood Problem

IoT sensors generate data continuously. A factory with 10,000 sensors reporting every second = 864 million readings per day. A smart city with millions of sensors: incomprehensible volume. Sending all raw data to the cloud is impractical.

Temperature Monitoring — With vs Without Preprocessing

Without preprocessing: Sensor sends 3,600 readings per hour to cloud. Enormous bandwidth, storage, and processing cost.
With preprocessing at gateway: Gateway computes hourly average and maximum, sends just 1 summary per hour. Same useful information. 99.97% less data transmitted.

Correct Answer: To reduce rapidly growing storage and processing costs

Preprocessing achieves: reduced bandwidth (less data sent), reduced storage (summaries not raw readings), reduced cloud compute (handle summaries not billions of points), and therefore significantly lower cloud bills.

Why Other Options Are Wrong

- "Data is already compressed" → Raw sensor data is not pre-compressed by default
- "Cloud is no longer needed" → Cloud still handles long-term storage and analysis

- "Preprocessing increases battery usage" → Actually saves battery by reducing wireless transmissions

2.9 Q9 — Virtualisation in Sensor Cloud: Virtual Sensors

Two Statements to Evaluate

Statement 1: Virtual sensors can be created by combining data from multiple physical sensors.
Statement 2: Multiple applications can share data from a single physical sensor.

▶ **Statement 1 — Virtual Sensors**

Weather Station Example

Physical sensors: temperature + humidity + pressure + wind speed.
Virtual sensor = "Comfort Index" combining all four readings into one software-derived value. No new hardware. The virtual sensor exists only in software, derived from real physical sensors.
Other examples: Air quality index (from CO2 + PM2.5 + humidity), fire risk index (from temperature + humidity + wind).

▶ **Statement 2 — Sensor Sharing**

One Weather Sensor, Four Applications

A single rooftop temperature sensor provides data simultaneously to: App 1 (smart thermostat) + App 2 (agricultural irrigation) + App 3 (weather dashboard) + App 4 (energy optimiser). One physical sensor, four completely different applications, all at the same time.

Correct Answer: Both statements are True

Virtual sensors enable higher-level derived data products without additional hardware. Sensor sharing maximises the return on investment from each physical sensor. Both are fundamental to sensor cloud virtualisation.

2.10 Q10 — Why Cloud Alone is Not Enough for Real-time IoT

The Fundamental Limitation: Speed of Light

No matter how powerful a cloud server is, it cannot overcome physics. Physical distance introduces an unavoidable minimum delay. The internet route from an IoT device to a cloud data centre involves multiple hops, each adding latency. Total round-trip: 100–600ms under good conditions.

Application	Max Acceptable Latency	Consequence of Delay
Self-driving car	<10ms	Collision at high speed
Industrial robot	<5ms	Equipment damage / injury
Cardiac monitor	<100ms	Delayed emergency response
Smart grid control	<20ms	Power grid instability

Correct Answer: Physical network distance causes high latency

Cloud data centres are geographically distant. The propagation delay this creates cannot be engineered away — it is a physical law. For applications requiring single-digit millisecond decisions, cloud-only architecture is physically impossible to make fast enough. Solution: Use Edge/Fog for real-time decisions, Cloud for everything else.

Real-Life Analogy

- Doctor in the same room: instant emergency response (Edge/Fog)
- Specialist in another city: 30 minutes for advice (Cloud)

For emergencies you need someone *in the room*. Cloud is the specialist for complex analysis, not the emergency responder.

Part B Quick Recap — All 10 Answers

Q1: Show all containers → `docker ps -a`
Q2: Volumes persist beyond container lifecycle (permanent storage)
Q3: Containers share OS kernel; many containers run inside one VM
Q4: Kubernetes does NOT do hardware virtualisation (hypervisor's job)
Q5: Green cloud = consolidate VMs onto fewer servers + power off idle machines
Q6: Sensors too weak for full virtualisation (limited CPU + memory)
Q7: Ultra-low latency IoT = edge/fog computing close to devices
Q8: Preprocess sensor data = reduce storage + bandwidth + cost
Q9: Virtual sensors + sensor sharing: both TRUE
Q10: Cloud alone = too far = high latency; need Edge + Cloud together

Master Summary — All 20 Questions

3.1 Part A: Mobile Cloud & Fog/Edge Computing

Q	Topic	Key Concept	Answer
1	Mobile Cloud Gaming Lag	Internet = most critical link	WAN dependency causes lag
2	Why Use Cloudlet?	Mini cloud close to user	Low-latency local computing
3	Useless Offloading Factor	Real-time resources matter	Number of installed apps
4	Cafeteria Server	Small local server near users	Cloudlet
5	Banking Security	Data travels phone to cloud	Confidentiality of data
6	Fog vs Cloud	Processing location matters	Fog = lower latency, real-time
7	5G + Edge Together	Two types of delay	Both delays reduced together
8	Satellite Image Processing	Too big for one processor	Partition + parallel processing
9	Geospatial Challenge	TBs or PBs of imagery	Storage + computation scale
10	Healthcare Monitoring	Must be instant response	Fog/Edge + 5G

3.2 Part B: Docker, Kubernetes & Green Cloud / Sensor Cloud

Q	Topic	Key Concept	Answer
1	Show All Containers	Running + stopped containers	`docker ps -a`
2	Docker Volumes	Storage outside container	Persist beyond container
3	Containers vs VMs	OS sharing architecture	Containers share OS kernel
4	Kubernetes Scope	K8s = container manager	Does NOT do hardware virtualisation
5	Green Cloud	Save energy in data centres	Consolidate VMs + power off idle servers
6	Sensor-Cloud Challenge	Sensors resource-constrained	Cannot support full virtualisation
7	Ultra-Low Latency IoT	Distance = delay	Edge/Fog near devices
8	Sensor Data Preprocessing	Too much raw data	Reduce storage + bandwidth cost
9	Virtual Sensors	Derived from multiple sensors	Both statements True
10	Cloud Alone Not Enough	Distance creates latency	High latency from network distance

The Golden Rules to Remember

Latency: Far = slow. Close = fast. Edge/Fog = close. Cloud = far. Use each appropriately.

Scale: Big data needs parallel processing. Never process massive datasets sequentially.

Energy: Consolidate workloads onto fewer servers. Power off idle ones.

Sensors: Small and weak = cannot virtualise. Use gateways. Preprocess to reduce data volume.

Docker: Container = temporary. Volume = permanent. `ps -a` = all containers.

Kubernetes: Manages containers. Hypervisor manages hardware. Different layers, different jobs.

5G + Edge: 5G reduces network delay. Edge reduces processing delay. Together = near-zero latency.